Homework 4:
Reinforcement Learning

I. Implementation details:

- Agent implementation:
    - Part 2:

```python
import random
class Agent:
    def __init__(self, k, epsilon):
        self.k = k
        self.epsilon = epsilon
        # Build a Q table to store the estimation
        self.q_table = [0] * self.k
        # Build a list to store the number of times each action has been taken
        self.action_count = [0] * self.k
    def select_action(self):
        """
        Choose an action based on the estimated expected reward for each action
        1. Apply epsilon greedy strategy
        2. Choose the action with the highest estimated expected reward
        3. If the random number is less than epsilon, choose a random action
        4. If the random number is greater than epsilon, choose the action with the highest estimated
expected reward
        5. Return the action
        """
        # Generate a random number between 0 and 1
        random_number = random.random()
        # Design the action in the range of 0 to k-1
        if random_number < self.epsilon:
            action = random.randint(0, self.k - 1)
        else:
            # max(self.q_table) will return the max value in the q_table, .index will return the index
of the first max value
            action = self.q_table.index(max(self.q_table))
        return action
    def update_q(self, action, reward):
        """
        Update the estimated expexted reward of the chossen action
        1. The estimated expected reward is updated using the formula:
        Q(a) = Q(a) + (reward - Q(a)) / N
        2. N is the number of times the action has been taken
        3. N is updated by adding 1 to the action count
        """
        # Update the action count by 1
        self.action_count[action] += 1
        # Update the estimated expexted reward using the formula
        # Q(a) = Q(a) + (reward - Q(a)) / N
        self.q_table[action] += (reward - self.q_table[action])/self.action_count[action]

    def reset(self):
        """
        Reset the agent
        """
        self.q_table = [0] * self.k
        self.action_count = [0] * self.k
```

- Part 6:

```python
class Agent:
    """
    1. If the alpha is None, the agent will be same as part2(), and the update_q method will use
    sample average method
    2. If the alpha is not None, the agent's update_q method will use the step-size update method
    3. The step-size update method will use the formula:
    Q(a) = Q(a) + alpha*(reward - Q(a)), alpha is the step-size
    """
    def __init__(self, k, epsilon, alpha=None):
        self.k = k
        self.epsilon = epsilon
        self.alpha = alpha
        self.q_table = [0] * self.k
        self.action_count = [0] * self.k
    def select_action(self):
        # random.random will pick a number between 0 to 1
        random_number = random.random()

        # If the randomnumber is less than epsilon, do the exloration
        if random_number < self.epsilon:
            action = random.randint(0, self.k-1)
            # Else, do the exploitation, which is choose the action with the highest q value
        else:
            action = self.q_table.index(max(self.q_table))
        return action
    def update_q(self, action, reward):
        # If alpha is None, use the sample average method
        if self.alpha == None:
            # Update the action count by 1
            self.action_count[action] += 1
            # Update the reward with the sample average method
            # The formula is Q(a) = Q(a) + (reward - Q(a)) / N
            self.q_table[action] += (reward - self.q_table[action]) / self.action_count[action]
        else:
            # Update the reward with the step-size update method
            # The formula is Q(a) = Q(a) + alpha*(reward-Q(a))
            self.q_table[action] += self.alpha*(reward-self.q_table[action])

    def reset(self):
        self.q_table = [0] * self.k
        self.action_count = [0] * self.k
```

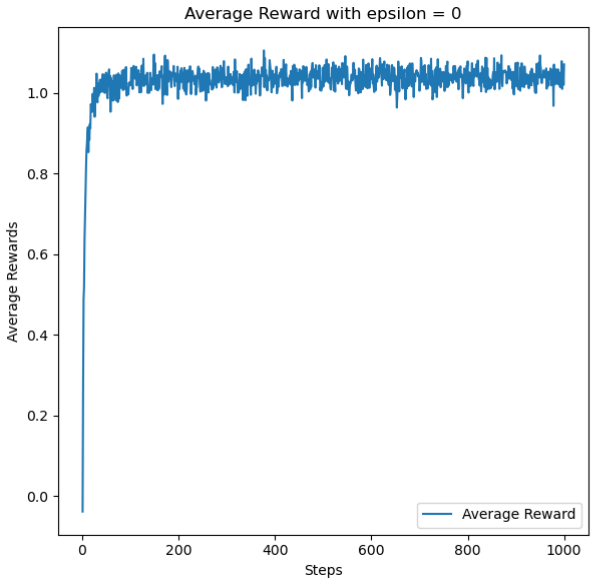
- Environment Implementation:
    - Part 1:

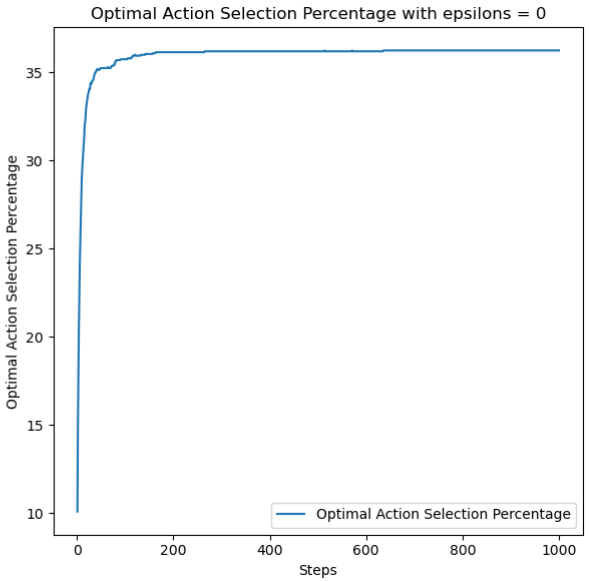
```python
import random
class BanditEnv:
    def __init__(self, n_arms):
        self.n_arms = n_arms
        self.means = None
        self.history = {"actions": [], "rewards": []}


    def reset(self):
        """
        Reset the game envitonment and the history
        """
        self.history = {"actions": [], "rewards": []}
        self.means = None



    def step(self, action):
        """
        Take an action and return the reward
        1. The reward distribution for each arm should be a Gaussian distribution with
        variance 1 and randomally generated mean
        2. The mean should be generated from a standard normal distribution
        """
        # Generate the means for each arm in standard normal distribution
        if self.means == None:
            self.means = [random.gauss(0, 1) for i in range(self.n_arms)]
        # Get the reward for the action based on the mean
        reward = random.gauss(self.means[action], 1)
        # Updqate the history
        self.history["actions"].append(action)
        self.history["rewards"].append(reward)
        return reward

    def export_history(self):
        """
        Export the action history and the reward history
        """
        return self.history
```

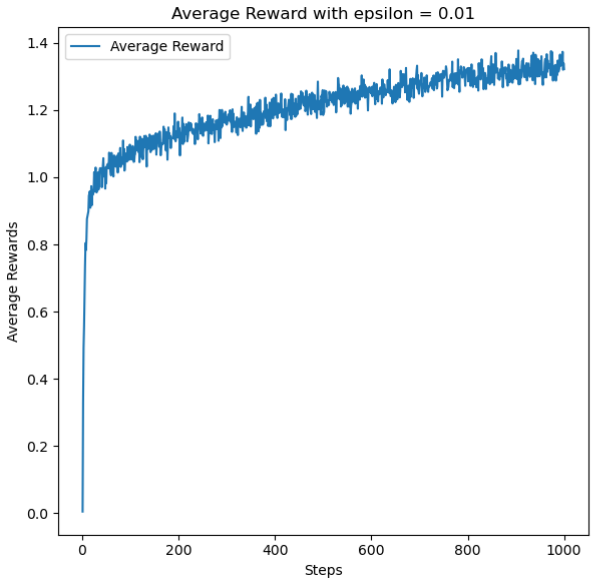
- Part 4:

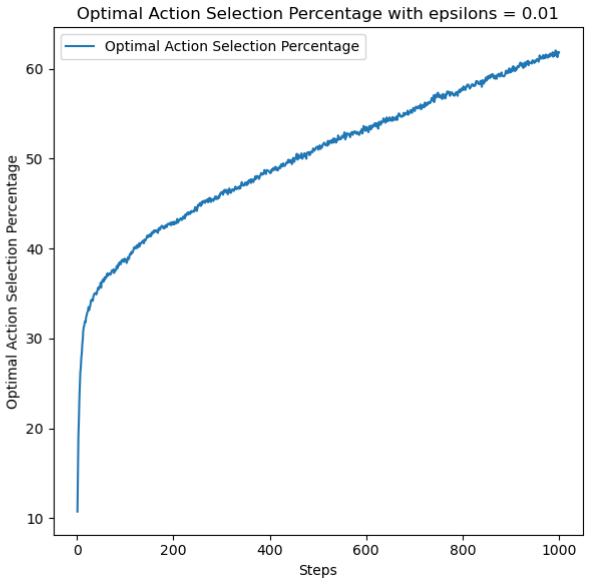
```python
class BanditEnv:
    """
    Create a non-stationary bandit environment that the mean reward
    of each action changes over time, in this case, the mean reward will change in each step
    1. Modify the BanditEnv class to include a non-stationary reward distribution
    2. The true mean of each action will change in every steps
    3. The true mean reward of each action will take independent random
    walks by adding a normally distributed increment with mean 0 and
    standard deviation 0.01 to all the true mean rewards
    """
    def __init__(self, n_arms, stationary):
        self.n_arms = n_arms
        self.stationary = stationary
        self.means = None
        self.history = {"actions": [], "rewards": []}

    def reset(self):
        """
        Reset the game envitonment and the history
        """
        self.history = {"actions": [], "rewards": []}
        self.means = None

    def step(self, action):
        # If is stationary, the mean reward will not change
        if self.means == None:
            self.means = [random.gauss(0, 1) for i in range(self.n_arms)]
        if self.stationary == True:
            # Get the reward for the action based on the mean
            reward = random.gauss(self.means[action], 1)
        else:
            self.means = [self.means[i] + random.gauss(0, 0.01) for i in range(self.n_arms)]
            reward = random.gauss(self.means[action], 1)

            # Update the history
        self.history["actions"].append(action)
        self.history["rewards"].append(reward)
        return reward
    def export_history(self):
        """
        Export the action history and the reward history
        """
        return self.history
```

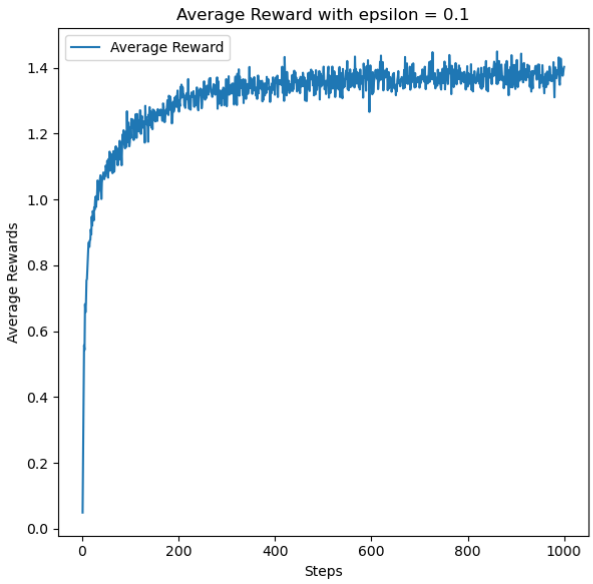
- Experiment Results
  - Part 3

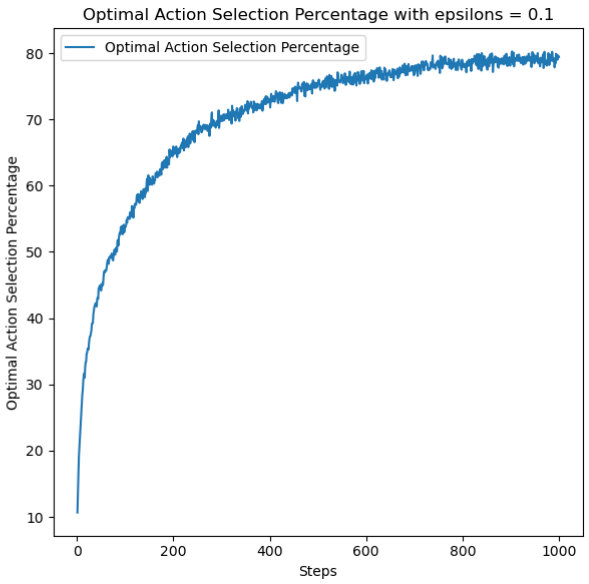






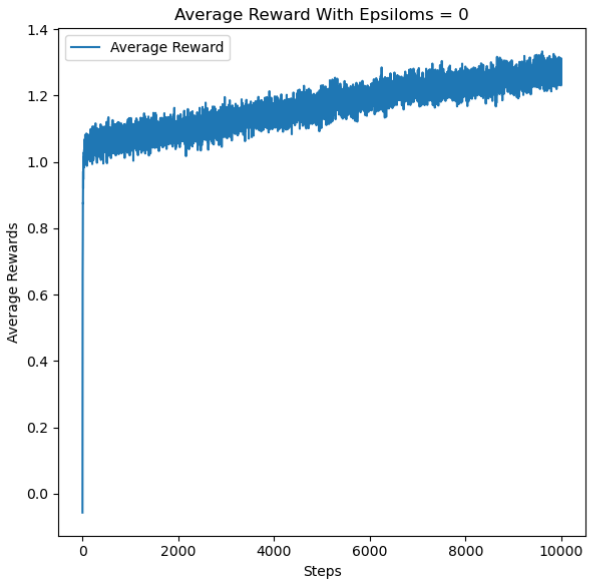


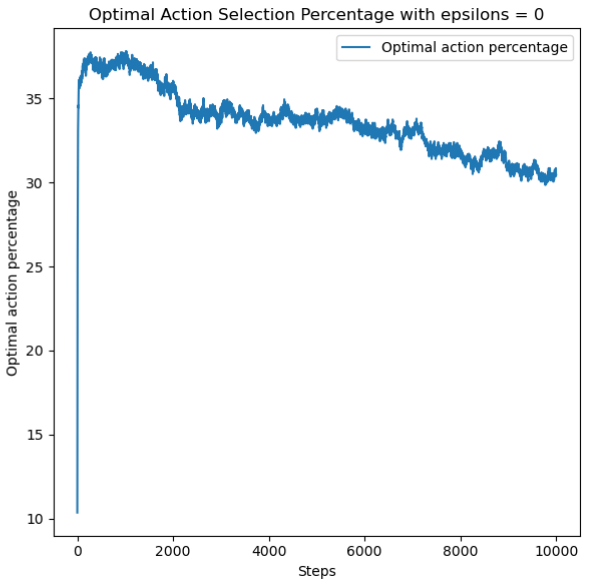


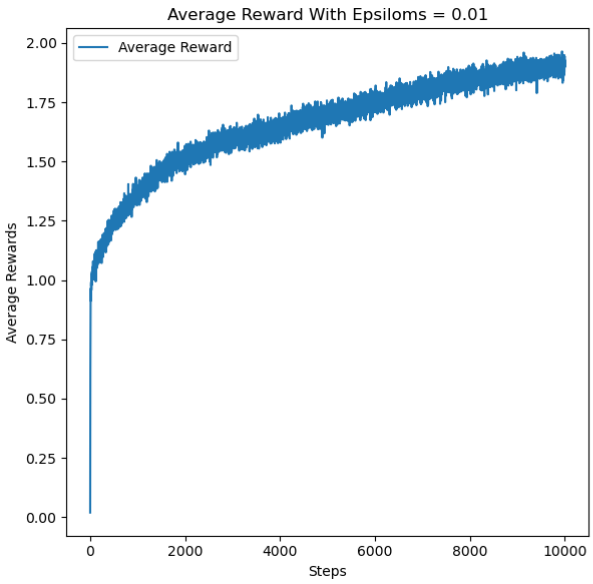


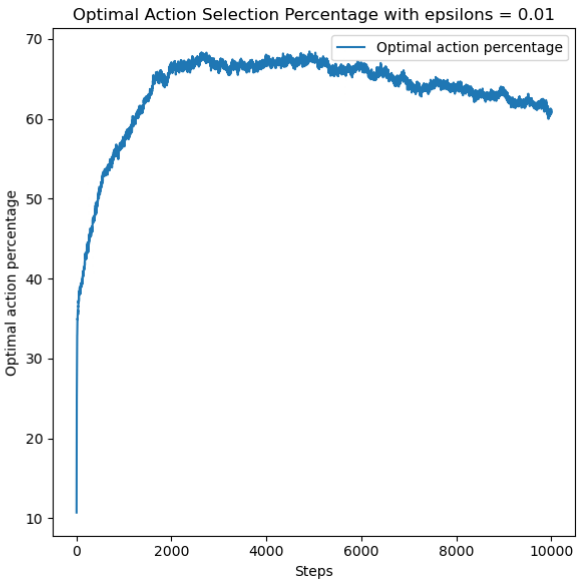
★ Analysis: Compare the result graph with different epsilon values, when epsilon=0,it’ s pure exploitation, so at the early stage the affection may be better, but since it doesn’ t explore new action, so it can’ t be an huge increasement at the end stage. On the other hand, we can compare epsilon=0.01 and epsilon=0.1,both of the graph have the increasement of the percentage of optimal action, epsilon=0.1 have more increasement than epsilon=0.01, and the average reward are better than rest of the two.

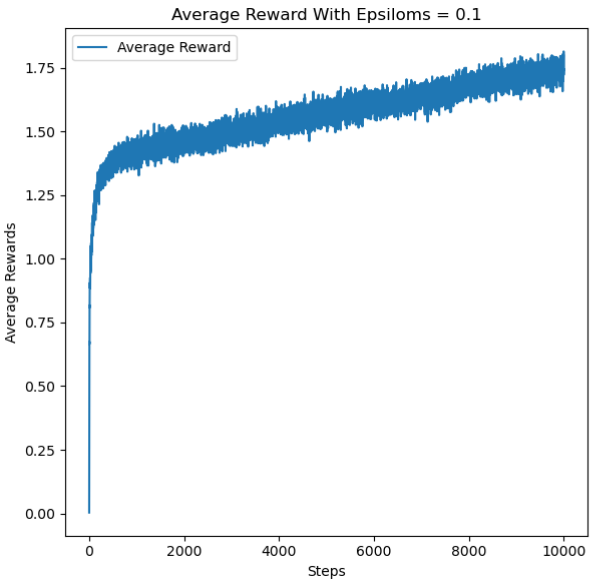
- Part 5

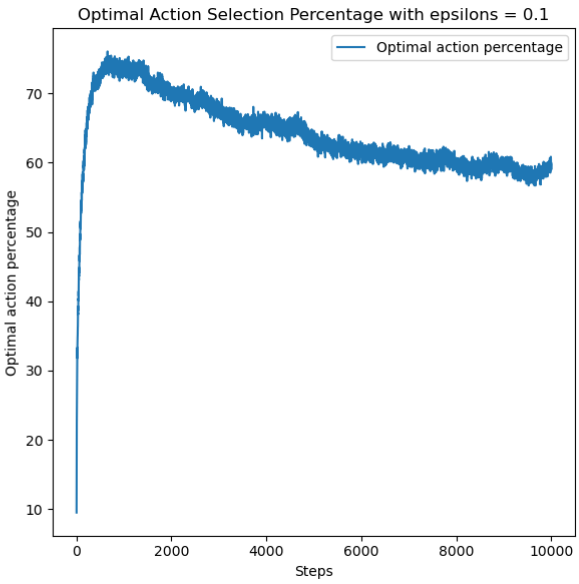





Average Reward With Epsiloms = 0.01
Average Reward
Average Rewards
Steps
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
0
2000
4000
6000
8000
10000


Optimal Action Selection Percentage with epsilons = 0.01
Optimal action percentage
Optimal action percentage
Steps
10
20
30
40
50
60
70
0
2000
4000
6000
8000
10000

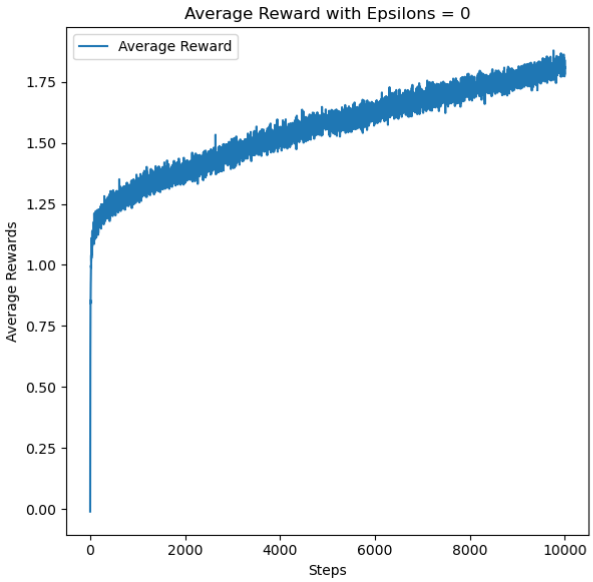

Average Reward With Epsiloms = 0.1
Average Reward
Average Rewards
Steps
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
0
2000
4000
6000
8000
10000

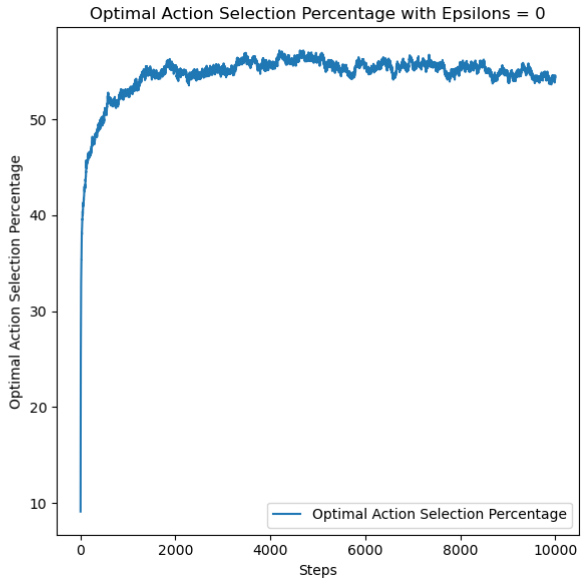

Optimal Action Selection Percentage with epsilons = 0.1
Optimal action percentage
Optimal action percentage
Steps
10
20
30
40
50
60
70
0
2000
4000
6000
8000
10000

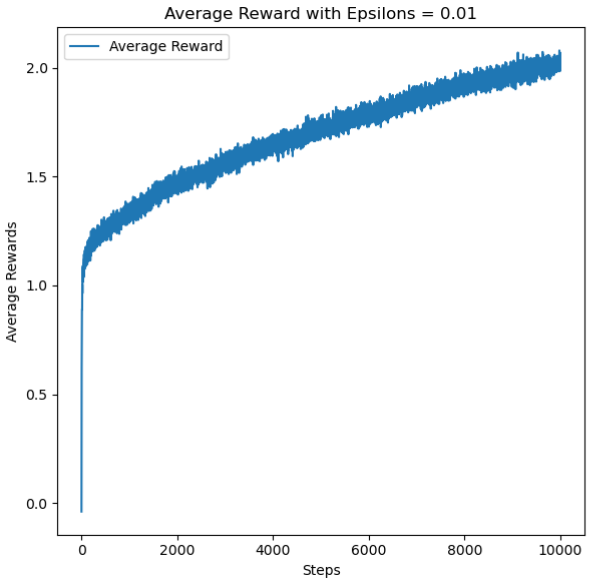
★ Analysis: We can compare the new environment with the old environment, the key difference between new and old is non-stationary or not, in the non-stationary environment, the reward will change in every step, so the q-value we update will also change in every step, which means it seems like the agent learn the strategy but when testing it, the accuracy will go down.

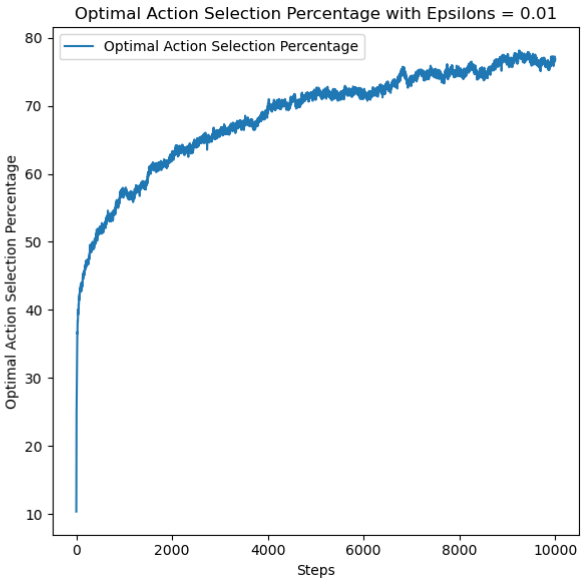
It also indicate the problem of the q-learning, it doesn’ t change the strategy with as the changing of the environment.

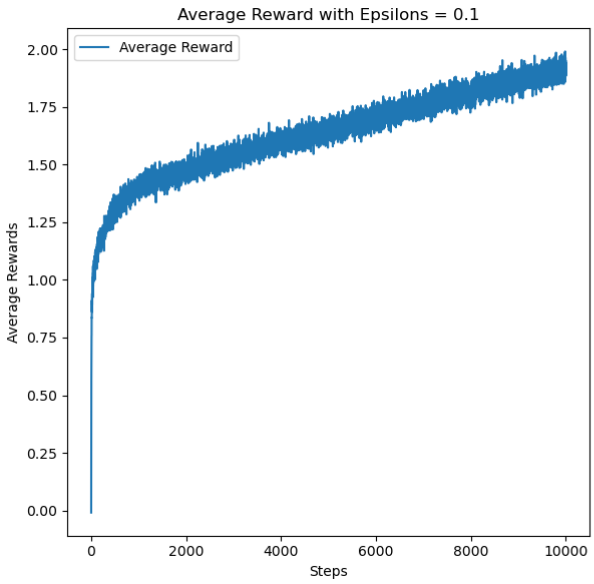
- Part 7

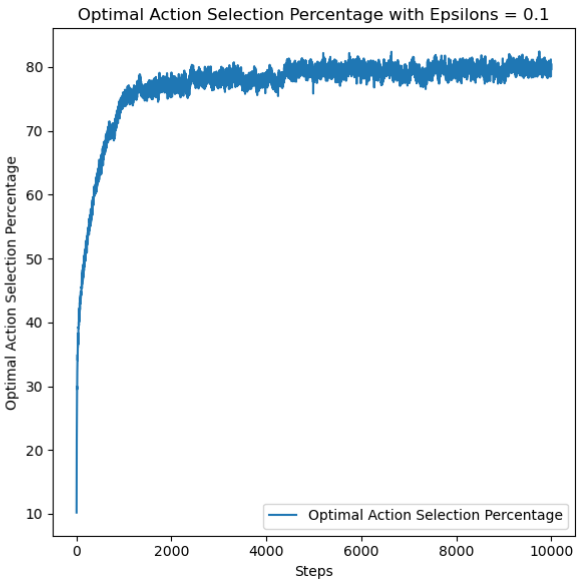

Average Reward with Epsilons = 0
Average Reward
Average Rewards
Steps
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
0
2000
4000
6000
8000
10000


Optimal Action Selection Percentage with Epsilons = 0
Optimal Action Selection Percentage
Optimal Action Selection Percentage
Steps
10
20
30
40
50
0
2000
4000
6000
8000
10000


Average Reward with Epsilons = 0.01
Average Reward
Average Rewards
Steps
0.0
0.5
1.0
1.5
2.0
0
2000
4000
6000
8000
10000


Optimal Action Selection Percentage with Epsilons = 0.01
Optimal Action Selection Percentage
Optimal Action Selection Percentage
Steps
10
20
30
40
50
60
70
80
0
2000
4000
6000
8000
10000





★ Analysis: In this part, we compare two study method, sample-average method and constant-size update method, the essence of the sample-average method is to take average of the history experience, this method have good affect on the stationary environment, but have bad affect on non-stationary environment, since the update process have slow reflection to the change of the environment, can not rapidly suit the changing of the reward, making the study policy outdate.

In contrast, constant-size update method use constant alpha value to let every new data have same importance, this make agent can't rapidly fix the error and revise the q-value to react to the change of the environment, we can compare part 5 and part 7 to see the affect of the difference of update_q method, can tell that at the end the percentage of optimal action is way higher using the constant-size update method than the sample-average method.